Lab 24.2– Replit for Mobile Applications

Name:MD.Jaffer shareef

Hno:2303A51496

TASK 1: ENVIRONMENT SETUP USING REPLIT

Aim:

To set up a cloud-based development environment using Replit for mobile application development.

Objective:

- To create a free Replit account
- To explore mobile app development templates (React Native / Flutter / HTML5 hybrid)
- To install required packages or dependencies

Create a free Replit account

Steps:

1. Open a web browser and go to Replit (https://replit.com).
2. Click on the **Sign Up** button.
3. Choose signup using Google, GitHub, or Email.
4. Enter required details and complete registration.
5. Verify your account if needed.
6. Log in to access the dashboard.

Explore mobile app development templates (React Native / Flutter / HTML5 hybrid)

Steps:

1. After login, click on **Create Repl**.
2. Browse available templates.
3. Select one of the following:
    - React Native (Node.js + Expo environment)
    - Flutter (if available)
    - HTML, CSS, JavaScript (Hybrid mobile apps)
4. Enter a project name.
5. Click **Create Repl**.
6. The development workspace will open.

Install required packages or dependencies in Replit environment

Steps:

1. Open the created project in Replit.
2. Navigate to the **Shell** section.
3. Install dependencies based on project type.

For React Native (Expo):

- Install Expo CLI:

- npm install -g expo-cli
- Create app:
- npx create-expo-app myApp
- Move into project:
- cd myApp
- Start project:
- npm start

For Node.js / Hybrid Apps:

- Install dependencies:
- npm install
- Install additional packages (if required):
- npm install express
- npm install cors

4. Wait for installation to complete.
5. Verify that dependencies are added successfully.
6. Click **Run** to test the environment.

Observation:

- Replit provides an online IDE with no local installation required.
- Multiple templates are available for mobile app development.
- Dependencies can be installed using simple commands.
- The platform provides a live preview for testing applications.

Result:

The Replit environment was successfully set up, templates were explored, and required dependencies were installed.

TASK 2: BASIC MOBILE APP

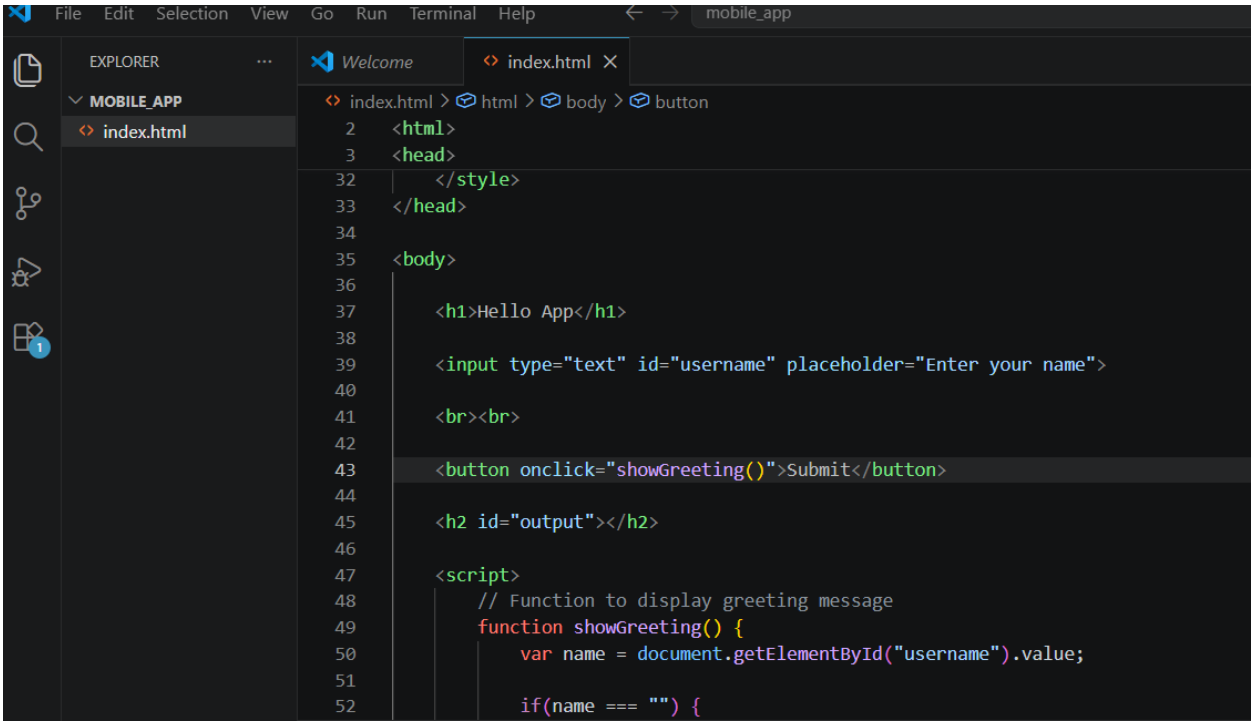
Prompt:

Create a simple mobile-friendly "Hello App" using Replit. The app should display a title, allow the user to enter their name, and show a greeting message when a button is clicked. Use AI assistance to generate the basic structure and improve the code.

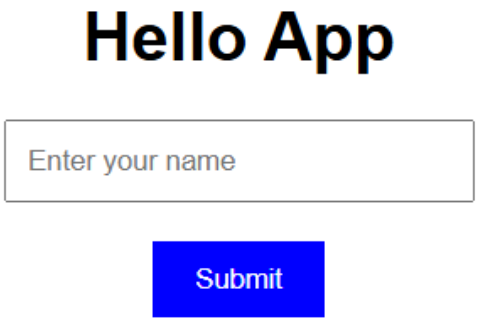
Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Hello App</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body style="text-align:center; font-family:Arial; margin-top:50px;">
    <h1>Hello App</h1>
    <input type="text" id="username" placeholder="Enter your name">
    <br><br>
```

```
    <button onclick="showGreeting()">Submit</button>

    <h2 id="output"></h2>

    <script>

        function showGreeting() {

            var name = document.getElementById("username").value;

            document.getElementById("output").innerText = "Hello, " + name + "!";

        }

    </script>


</body>

</html>
```



Output:


Hello App
Enter your name
Submit

Hello App
john
Submit
Hello, john!

Observation:

While doing this task, I learned how to create a simple mobile app that takes user input and shows output instantly. It was easy to understand how the button click triggers the function and displays the greeting message. Using AI tools helped me write the basic code quickly and correct mistakes. Replit made it simple to run and test the app without installing anything. Overall, this task improved my understanding of basic app development and user interaction..

TASK 3: ADDING FEATURES WITH AI ASSISTANCE

Prompt:

Enhance the existing Hello App by adding a navigation bar (Home, About, Contact), a simple user registration form, and basic styling using CSS. Use AI assistance in Replit to improve the code and add inline comments explaining each function.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Enhanced Hello App</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <style>
```

```
/* General body styling */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    text-align: center;
    background-color: #f4f4f4;
}
/* Navigation bar styling */
nav {
    background-color: #333;
    padding: 12px;
}
nav a {
    color: white;
    margin: 0 15px;
    text-decoration: none;
    font-weight: bold;
}
nav a:hover {
    text-decoration: underline;
}
/* Container for the form */
.container {
    margin-top: 40px;
```

```
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    display: inline-block;
    box-shadow: 0px 0px 10px #ccc;
}
/* Input and button styling */
input, button {
    padding: 10px;
    margin: 10px 0;
    width: 80%;
    max-width: 300px;
    border-radius: 5px;
    border: 1px solid #ccc;
}
button {
    background-color: #333;
    color: white;
    border: none;
    cursor: pointer;
}
button:hover {
    background-color: #555;
}
```

```
        /* Output message styling */
        #result {
            margin-top: 20px;
            color: green;
        }
    </style>
</head>
<body>
    <!-- Navigation Bar -->
    <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
    <div class="container">
        <h1>Hello App – User Registration</h1>
        <!-- Input field for user's name -->
        <input type="text" id="name" placeholder="Enter your name"><br>
        <!-- Input field for user's email -->
        <input type="email" id="email" placeholder="Enter your email"><br>
        <!-- Button to trigger registration -->
        <button onclick="registerUser()">Register</button>
        <!-- Output message area -->
        <h2 id="result"></h2>
```

```
    </div>
    <script>
        // Function to register a user and display confirmation
        function registerUser() {
            // Get the name and email input values
            let name = document.getElementById("name").value;
            let email = document.getElementById("email").value;
            // Check if inputs are not empty
            if(name === "" || email === "") {
                document.getElementById("result").innerText = "Please fill in all fields.";
                document.getElementById("result").style.color = "red";
                return;
            }
            // Display a success message with user's name and email
            document.getElementById("result").innerText =
                "Registration Successful! Welcome, " + name + " (" + email + ")";
            // Clear input fields after registration
            document.getElementById("name").value = "";
            document.getElementById("email").value = "";
        }
    </script>
</body>
</html>
```

EXPLORER

MOBILE_APP

index.html

Welcome | index.html

index.html > html > head > style

```html
<html>
<head>

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            text-align: center;
            background-color: #f4f4f4;
        }

        nav {
            background-color: #333;
            padding: 12px;
        }

        nav a {
            color: white;
            margin: 0 15px;
            text-decoration: none;
            font-weight: bold;
        }
```

EXPLORER

MOBILE_APP

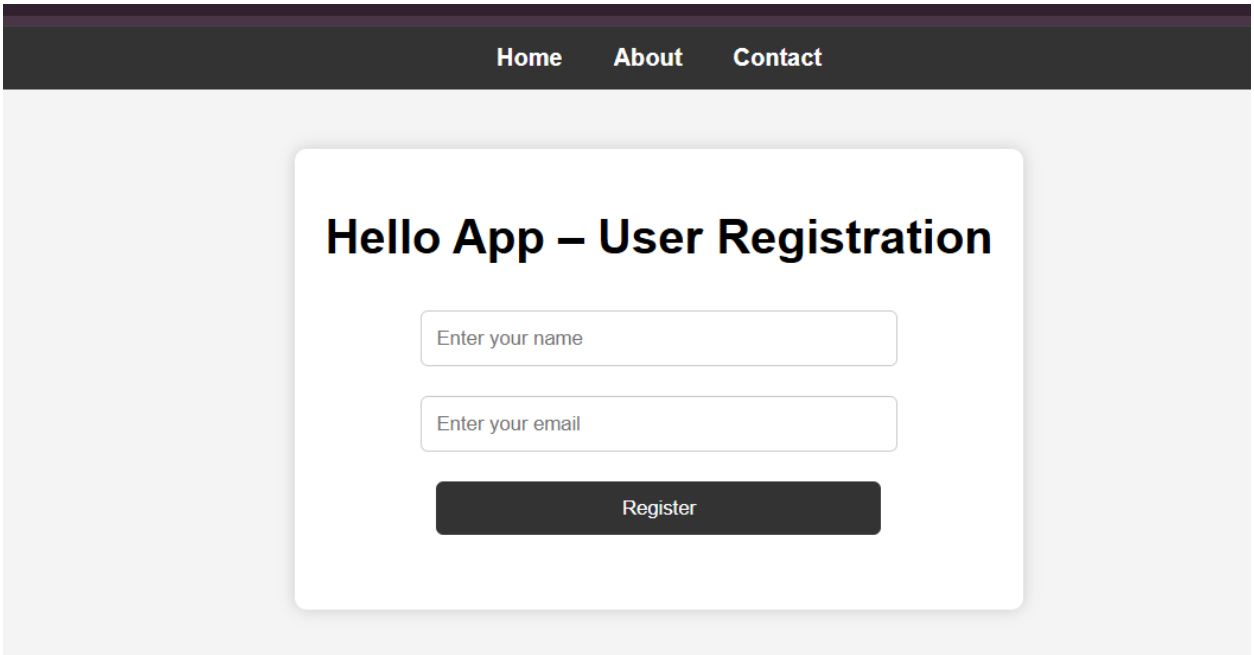
index.html

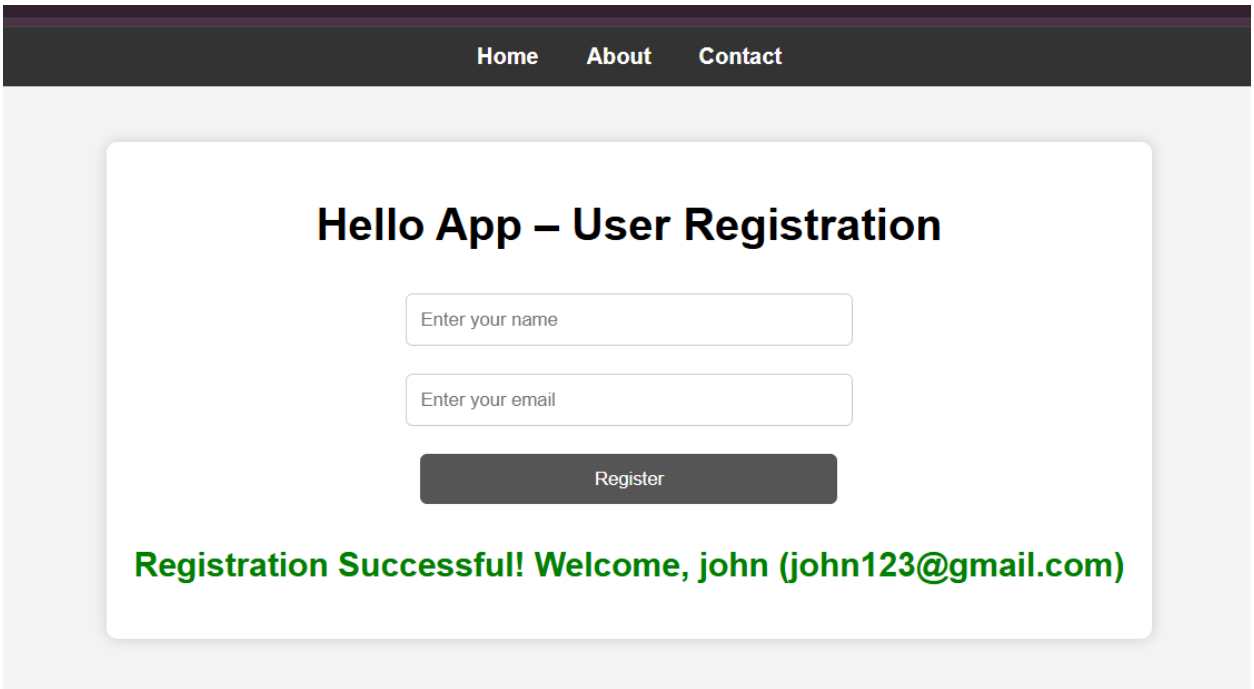
Welcome | index.html

index.html > html > head > style

```html
<html>
<body>
<nav>
    <a href="#">Contact</a>
</nav>

<div class="container">

    <h1>Hello App - User Registration</h1>

    <input type="text" id="name" placeholder="Enter your name"><br>

    <input type="email" id="email" placeholder="Enter your email"><br>

    <button onclick="registerUser()">Register</button>

    <h2 id="result"></h2>

</div>

<script>
function registerUser() {

    let name = document.getElementById("name").value;
```

Output:


Home
About
Contact
Hello App – User Registration
Enter your name
Enter your email
Register


Home
About
Contact
Hello App – User Registration
Enter your name
Enter your email
Register
Registration Successful! Welcome, john (john123@gmail.com)

Observation:

In this task, I learned how to improve a basic app by adding more features like navigation and form handling. It was interesting to see how user details can be collected and displayed. Styling made the app look more organized and user-friendly. With AI help, adding comments and understanding the code became easier. This task gave me a better idea of how real applications are built step by step.

TASK 4: TESTING ON MOBILE

Run the app in Replit's mobile preview mode.

Steps:

1. Open the Replit project with your enhanced Hello App.
2. Click on the **"Run"** button to start the app.
3. Select the **mobile preview option** in Replit to see how the app looks on a smartphone screen.
4. The preview simulates a mobile device so you can interact with the app just like on a real phone.

Test responsiveness on different screen sizes.

Steps:

1. Resize the browser window to simulate tablets, phones, and desktops.
2. Observe how the navigation bar, input fields, buttons, and messages adjust automatically.
3. Check that text does not overlap and elements remain cantered.

4. Test on multiple devices if available (phone, tablet, laptop) to confirm the layout works well everywhere.

Fix UI alignment issues with AI suggestions.

Steps:

- Replit AI (Ghostwriter) is enabled in the workspace.
- AI assistance is used to identify UI alignment issues.
- Suggested changes in CSS (margin, padding, width, font-size) are applied.
- The application is re-run to verify improvements.
- Adjustments are repeated until the UI becomes responsive and well-aligned.

Observation:

While testing the app, I noticed that some input fields and buttons were not perfectly aligned on smaller screens. Using Replit's mobile preview helped me identify these issues quickly. AI suggestions made it easier to fix spacing, margins, and font sizes. After applying the fixes, the app became fully responsive, user-friendly, and visually neat. I learned how important testing on different screen sizes is and how AI can save time in correcting UI problems.

Result:

The enhanced Hello App is fully mobile responsive. Navigation bar, registration form, buttons, and messages display correctly on all screen sizes. AI-assisted suggestions helped fix alignment issues efficiently and improved the app's overall appearance.

TASK 5: DEPLOYMENT

Deploy the app using Replit hosting

Steps:

- The completed Hello App project is opened in Replit.
- The **Run** button is clicked to ensure proper functioning.
- The **Share / Publish** option is selected from the top-right corner.
- A public URL is generated by Replit.
- The application is accessed through the generated link.

Export the app to a mobile framework like Expo (React Native)

Steps:

1. Ensure your project uses a React Native template.
2. Open Replit's shell and install Expo CLI:

   npm install -g expo-cli

3. Initialize an Expo project:

```
npx create-expo-app MyApp
cd MyApp
npm start
```

4. Scan the QR code in Replit with **Expo Go** on your mobile device.
5. Share the Expo project link or QR code with peers for testing on mobile devices.

Share the deployment link with peers for testing.

Steps:

1. Copy the Replit public URL or Expo project link.
2. Send it via email, chat, or classroom platform.
3. Peers can open the link on any browser or mobile device to interact with the app.

Observation:

Deploying the Hello App made it accessible to peers for testing without requiring any installation. The live preview demonstrated the app's responsiveness and user interaction. Using Replit hosting and Expo showed how apps can go from development to real-world usage quickly. AI-assisted inline comments and debugging made the code easier to maintain and deploy. Overall, this step highlighted the importance of deployment for collaboration and feedback.

Result:

- The Hello App is fully deployed and accessible via a live link.
- The app functions correctly on both desktop and mobile devices.

- Deployment and sharing improved understanding of cloud hosting, mobile testing, and real-world app usage.